Φυλλάδιο θεωρίας - Booleans, Τελεστές Σύγκρισης, Λογικοί Τελεστές, οι Εντολές if, elif, else, match.

Boolean μεταβλητές

Τι είναι τα Boolean;

Στον προγραμματισμό, πολλές φορές θέλουμε ο υπολογιστής να παίρνει αποφάσεις:

Αν κάτι είναι αληθινό ➜ κάνε Α
αλλιώς, αν είναι ψεύτικο ➜ κάνε Β.

Για να μπορέσουμε να εκφράσουμε αυτή τη διαφορά ανάμεσα στο αληθινό και το ψεύτικο, η Python χρησιμοποιεί έναν ειδικό τύπο δεδομένων που ονομάζεται Boolean.

Ο τύπος δεδομένων bool

Ένας Boolean μπορεί να έχει μόνο δύο τιμές:

```
True     # Αληθές
False    # Ψευδές
```

Προσοχή! Πρέπει το πρώτο γράμμα να είναι κεφαλαίο.

Μπορούμε να θέσουμε λοιπόν μια Boolean μεταβλητή την τιμη True ή False

Παράδειγμα:

```
is_sunny = True
is_rainy = False
```

Σκέψου το Boolean ως έναν «ανιχνευτή αλήθειας»:

- ✓ Αν κάτι συμβαίνει, η Python λέει True.
- ✓ Αν δεν συμβαίνει, λέει False.

Τελεστές Σύγκρισης (Comparison operators)

Οι συγκριτικοί τελεστές είναι σύμβολα που χρησιμοποιούμε στην Python για να συγκρίνουμε δύο τιμές (αριθμούς, μεταβλητές, ή ακόμα και λέξεις).

Το αποτέλεσμα της σύγκρισης είναι πάντα Boolean, δηλαδή True ή False.

Οι βασικοί συγκριτικοί τελεστές

Τελεστής	Ανάγνωση στα ελληνικά	Παράδειγμα	Αποτέλεσμα
`==`	ίσο με	`5 == 5`	**True**
`≠`	διάφορο από	`5 ≠ 3`	**True**
`>`	μεγαλύτερο από	`10 > 7`	**True**
`<`	μικρότερο από	`3 < 2`	**False**
`⩾`	μεγαλύτερο ή ίσο από	`4 ⩾ 4`	**True**
`⩽`	μικρότερο ή ίσο από	`2 ⩽ 5`	**True**

Παραδείγματα:

```
a = 10
b = 5
print(a > b)
print(a == b)
print(a ≠ b)
print(a ⩽ 10)
```

Output:

```
True
False
True
True
```

Ειδική σημείωση για κείμενα (strings)

Στην Python χρησιμοποιώντας αυτούς τους τελεστές μπορούμε να συγκρίνουμε και λέξεις (strings).Η Python τότε θα συγκρίνει τα γράμματα αλφαβητικά (λεξικογραφικά, ποιο γράμμα εμφανίζεται πρώτο στην αλφάβητο).

Παράδειγμα:

```
print("apple" == "apple")
print("apple" < "bowl")
print("bowl" > "candy")
```

Output:

```
True
True
False
```

Δηλαδή «apple» < «bowl» επειδή το “a” προηγείται του “b” στο αλφάβητο, ενώ ο τελεστής “==” θα συγκρίνει αν οι δύο λέξεις είναι ίδιες.

Chaining Comparison Operators (Αλυσιδωτές Συγκρίσεις)

Μπορούμε επίσης στην Python να ελέγχουμε εάν μια μεταβλητή βρίσκεται μέσα σε ένα διάστημα χρησιμοποιώντας αλυσιδωτές συγκρίσεις.

Παράδειγμα:

```
x = 5

print(1 < x < 10)
```

Output:

```
True
```

! Είναι σαν να ελέγχουμε εάν δύο συγκρίσεις ισχύουν ταυτόχρονα, x>1 ΚΑΙ x<10, δηλαδή αν το x ανήκει στο διάστημα ενδιάμεσα στο 1 και το 10, όπως κάνουμε στα μαθηματικά.

Λογικοί Τελεστές (Logical Operators)

Οι **λογικοί τελεστές** χρησιμοποιούνται για να **συνδυάζουμε** ή να **αντιστρέφουμε** συνθήκες (δηλαδή εκφράσεις που δίνουν True ή False).

Με αυτούς μπορούμε να ελέγξουμε **πολλά πράγματα ταυτόχρονα** και να πάρουμε μία τελική απάντηση, όταν χρειάζεται να ελέγξουμε πιο περίπλοκες συνθήκες.

Οι βασικοί λογικοί τελεστές στην Python:

Τελεστής	Ανάγνωση	Περιγραφή	Παράδειγμα	Αποτέλεσμα
and	και	Επιστρέφει True μόνο αν και οι δύο συνθήκες είναι True	`x < 10 and x > 0`	True αν το x είναι μεταξύ 0 και 10
or	ή	Επιστρέφει True αν τουλάχιστον μία συνθήκη είναι True	`x < 0 or x > 100`	True αν το x είναι έξω από 0–100
not	όχι	Αντιστρέφει το αποτέλεσμα: True γίνεται False, False γίνεται True	`not(x > 5)`	True αν το x δεν είναι πάνω από 5

Παραδείγματα:

```
x = 5

print(x > 2 and x < 10)
print(x > 2 or x > 10)
print(not(x > 2 and x < 10))
```

Output:

```
True
True
False
```

Οι λογικοί τελεστές λειτουργούν όπως η **κοινή λογική** που χρησιμοποιούμε καθημερινά.

Κατάσταση στην καθημερινή ζωή	Παράδειγμα στην Python
Αν **έχω λεφτά** και **είναι ανοιχτό το μαγαζί**, θα αγοράσω τσίχλες.	`if have_money and shop_open:`
Αν **είναι Σάββατο** ή **είναι Κυριακή**, δεν έχω σχολείο.	`if day == "Saturday" or day == "Sunday":`
Αν **δεν** είμαι άρρωστος, θα πάω στη συναυλία.	`if not sick:`

Με αυτόν τον τρόπο μπορούμε να εισάγουμε λογική στα προγράμματά μας.

Ας δούμε τους τρεις τελεστές λίγο πιο αναλυτικά:

Ο τελεστής and

Συνθήκη A	Συνθήκη B	A and B
True	True	✓ True
True	False	✗ False
False	True	✗ False
False	False	✗ False

Επιστρέφει True **μόνο** αν και οι δύο συνθήκες είναι True.

Αν έστω μία είναι False, το αποτέλεσμα είναι False.

Ο τελεστής or

Συνθήκη A	Συνθήκη B	A or B
True	True	✓ True
True	False	✓ True
False	True	✓ True
False	False	✗ False

Επιστρέφει True αν **τουλάχιστον μία** συνθήκη είναι True.

Μόνο αν **όλες** είναι False, τότε το αποτέλεσμα είναι False.

Ο τελεστής not

Συνθήκη A	not A
True	✗ False
False	✓ True

Αντιστρέφει το αποτέλεσμα μιας συνθήκης.

not True → False
not False → True

Συνδυασμός λογικών τελεστών

Μπορούμε να συνδυάσουμε περισσότερους από έναν λογικούς τελεστές μέσα στην ίδια συνθήκη.

Παράδειγμα:

```
age = 14
has_ticket = True
print( (age ⩾ 12 and age ⩽ 17) and has_ticket)
```

Output:

```
True
```

Παράδειγμα:

```
day = "Saturday"
good_weather = True

print( (day == "Saturday" or day == "Sunday") and  good_weather)
```

Output:

```
True
```

Εντολές If, Elif, Else

Η Python μας επιτρέπει να παίρνουμε αποφάσεις με τις εντολές if, elif και else. Αυτές ελέγχουν συνθήκες (που δίνουν True ή False) και εκτελούν διαφορετικό κώδικα ανάλογα με το αποτέλεσμα.

Η εντολή if

Η εντολή if εκτελεί ένα μπλοκ εντολών μόνο αν μια συνθήκη είναι αληθής (True). Αν η συνθήκη είναι ψευδής (False), ο υπολογιστής απλώς προσπερνά αυτό το μπλοκ.

Σύνταξη

```
if <συνθήκη>:
    # εντολές που θα εκτελεστούν αν η συνθήκη είναι True
```

Προσοχή στην εσοχή (indentation): Όλες οι εντολές που ανήκουν στο if πρέπει να γράφονται πιο μέσα.

Παράδειγμα:

```
a = 3
b = 200
if b > a:
    print("b είναι μεγαλύτερο από a")
```

Output:

```
b είναι μεγαλύτερο από a
```

Εδώ η συνθήκη b > a είναι True, οπότε η Python εκτελεί την print(). Αν δεν ήταν, θα αγνοούσε το μπλοκ του if και δεν θα εμφανιζόταν τίποτα στο output.

Προσοχή: Εάν δεν είχαμε κρατήσει την εσοχή στο print κάτω από το if θα παίρναμε σφάλμα, καθώς δεν θα υπήρχε κώδικας να εκτελεστεί αν η συνθήκη είναι True.

```
a = 3
b = 200
if b > a:
print("b είναι μεγαλύτερο από a")
```

```
ERROR!
```

Εάν θέλουμε να εκτελεστούν πολλές εντολές αν μία συνθήκη είναι αληθής θα πρέπει να προσέξουμε να κρατήσουμε την ίδια εσοχή για όλες τις εντολές.

Παράδειγμα:

✓ Σωστή σύνταξη:

```
age = 20
if age ⩾ 18:
  print("Είσαι ενήλικας")
  print("Μπορείς να ψηφίσεις")
```

× Λανθασμένη σύνταξη:

```
age = 20
if age ⩾ 18:
  print("Είσαι ενήλικας")
print("Μπορείς να ψηφίσεις")
```

Στην δεύτερη περίπτωση μέσα στην συνθήκη του if ανήκει μόνο το πρώτο print!

Η εντολή elif

Αν θέλουμε να ελέγξουμε περισσότερες από μία συνθήκες, μπορούμε να χρησιμοποιήσουμε elif (προέρχεται από το else if).

Η Python θα ελέγξει τις συνθήκες μία προς μία, από πάνω προς τα κάτω:

- Μόλις βρει μία που είναι True,
- Εκτελεί το αντίστοιχο μπλοκ,
- Και αγνοεί τα υπόλοιπα.

Σύνταξη

```
if <συνθήκη1> :
    # εντολές αν η συνθήκη1 είναι True
elif <συνθήκη2> :
    # εντολές αν η συνθήκη2 είναι True
elif <συνθήκη3> :
    # εντολές αν η συνθήκη3 είναι True
…
```

Μπορούμε να βάζουμε όσες elif συνθήκες χρειάζεται η εφαρμογή μας, κρατώντας αυτή τη σύνταξη.

Παράδειγμα:

```
a = 33
b = 33

if b > a:
    print("b είναι μεγαλύτερο από a")
elif a == b:
    print("a και b είναι ίσα")
```

Output:

```
a και b είναι ίσα
```

Η πρώτη συνθήκη b > a είναι False, οπότε η Python ελέγχει τη δεύτερη (a == b), που είναι True. Αμέσως εκτελεί το print() και σταματάει τον έλεγχο.

> **Προσοχή:** Εάν παραπάνω συνθήκες από μία είναι True τότε η Python θα εκτελέσει κώδικα για την πρώτη συνθήκη που θα συναντήσει True κοιτώντας από πάνω προς τα κάτω.

Παράδειγμα:

```
x = 10

if x>1:
     print(“hello1”)
elif x>2:
     print(“hello2”)
```

Output:

```
hello1
```

Όπως βλέπουμε στον παραπάνω κώδικα παρότι και η δεύτερη συνθήκη x>2 είναι True ο κώδικάς της θα προσπεραστεί και δεν θα εκτελεστεί ποτέ. Για να εκτελεστεί θα έπρεπε να χρησιμοποιήσουμε δύο if συνθήκες.

Χρησιμοποιούμε elif όταν ελέγχουμε αμοιβαία αποκλειόμενα γεγονότα, οπότε είναι True μία συνθήκη!

Η εντολή else

Η else εκτελείται όταν καμία από τις προηγούμενες συνθήκες δεν είναι True.

Είναι το «όλα τα υπόλοιπα» ή «σε κάθε άλλη περίπτωση».

Σύνταξη

```
if <συνθήκη1>:
    # εντολές αν η συνθήκη1 είναι True
elif <συνθήκη2>:
    # εντολές αν η συνθήκη2 είναι True
else:
    # εντολές αν όλες οι παραπάνω είναι False
```

Παράδειγμα:

```
a = 200
b = 33

if b > a:
    print("b είναι μεγαλύτερο από a")
elif a == b:
    print("a και b είναι ίσα")
else:
    print("a είναι μεγαλύτερο από b")
```

Output:

```
a είναι μεγαλύτερο από b
```

i Εδώ καμία από τις δύο πρώτες συνθήκες δεν είναι αληθής, οπότε η Python φτάνει στο else και εκτελεί τις εντολές του.

Μπορούμε να μην χρησιμοποιήσουμε την ενδιάμεση elif συνθήκη αν δεν το χρειαζόμαστε.

Σύντομη μορφή if (Short Hand If)

Αν η εντολή που θέλουμε να εκτελέσουμε είναι μία μόνο γραμμή, μπορούμε να γράψουμε το if σε μία γραμμή.

Σύνταξη

```
if <συνθήκη> : <εντολή>
```

Παράδειγμα:

```
a = 200
b = 33
if a > b: print("a είναι μεγαλύτερο από b")
```

Output:

```
a είναι μεγαλύτερο από b
```

Σύντομη μορφή if...else (Short Hand If...Else)

Μπορούμε επίσης να γράψουμε if και else στην ίδια γραμμή, όταν είναι απλές εντολές.

Σύνταξη

```
<εντολή-if> if <συνθήκη> else <εντολή-else>
```

Παράδειγμα:

```
a = 2
b = 330
print("A") if a > b else print("B")
```

Output:

```
B
```

Αυτός ο τρόπος λέγεται conditional expression και είναι χρήσιμος για απλά “ναι ή όχι” αποτελέσματα.

Εμφωλευμένα if (Nested if)

Μπορούμε να βάλουμε ένα if μέσα σε ένα άλλο if. Αυτό λέγεται εμφωλευμένο if (nested if) και μας επιτρέπει πιο σύνθετες αποφάσεις.

Σύνταξη

```
if <συνθήκη1> :
        <εντολή1>
        if <συνθήκη2>:
               <εντολή2>
```

Παράδειγμα:

```
x = 41

if x > 10:
    print("Πάνω από 10")
    if x > 20:
        print("Επίσης πάνω από 20!")
    else:
        print("Αλλά όχι πάνω από 20.")
```

Output:

```
Πάνω από 10
Επίσης πάνω από 20!
```

Εδώ έχουμε ένα if μέσα σε ένα άλλο. Η Python πρώτα ελέγχει αν x > 10, και μέσα σε αυτό το μπλοκ ελέγχει αν x > 20.

Συνοπτικός Πίνακας με όσα είδαμε ως τώρα:

Εντολή	Ρόλος	Εκτελείται όταν...
if	Αρχικός έλεγχος	Η συνθήκη είναι True
elif	Εναλλακτικός έλεγχος	Η προηγούμενη ήταν False και αυτή είναι True
else	Τελική περίπτωση	Όλες οι προηγούμενες ήταν False
nested if	Εμφωλευμένος έλεγχος	Μια δεύτερη απόφαση μέσα σε άλλη
short hand	Σύντομη μορφή	Όταν έχουμε μικρές, μονογραμμικές εντολές

Η εντολή Match

Η εντολή match στην Python χρησιμοποιείται για να επιλέξουμε ποια από πολλές επιλογές θα εκτελεστεί, με βάση την τιμή μιας έκφρασης.

Αντί να γράφουμε πολλές if...elif...else, μπορούμε να χρησιμοποιήσουμε match για πιο καθαρό και οργανωμένο κώδικα.

Σύνταξη

```
match <έκφραση>:
    case <τιμή1>:
        # μπλοκ εντολών αν έκφραση == τιμή1
    case <τιμή2>:
        # μπλοκ εντολών αν έκφραση == τιμή2
    case _:
        # μπλοκ εντολών αν καμία από τις παραπάνω δεν ταιριάζει
```

Η _ χρησιμοποιείται ως «προεπιλεγμένη περίπτωση» (default) αν δεν γίνει κανένα match.

Παραδείγματα:

Πράδειγμα1:

```
day = 4
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case 4:
        print("Thursday")
    case 5:
        print("Friday")
    case 6:
        print("Saturday")
    case 7:
        print("Sunday")
```

Output:

```
Thursday
```

Παράδειγμα 2 (συνδυασμός τιμών):

```
day = 4
match day:
    case 1 | 2 | 3 | 4 | 5:
        print("Today is a weekday")
    case 6 | 7:
        print("I love weekends!")
```

Output:

```
Today is a weekday
```

Εδώ ο τελεστής “|” είναι σαν τον τελεστή or.

Παράδειγμα 3 (guard στο case):

```
month = 5
day = 4
match day:
    case 1 | 2 | 3 | 4 | 5 if month == 4:
        print("A weekday in April")
    case 1 | 2 | 3 | 4 | 5 if month == 5:
        print("A weekday in May")
    case _:
        print("No match")
```

Output:

```
A weekday in May
```

Προσοχή: Όπως είπαμε ήδη για τα elif, μόλις βρεθεί το πρώτο match θα εκτελεστεί και οι υπόλοιπες περιπτώσεις θα προσπεραστούν! Χρειάζεται λοιπόν προσοχή στην σειρά που τοποθετούμε τις συνθήκες.

Σκέψου την εντολή match σαν ένα μεγάλο διακόπτη επιλογών: ελέγχει την τιμή της έκφρασης μία φορά και «πηδάει» κατευθείαν στην κατάλληλη επιλογή.